

Gradient optimization methods

Théodore Cherrière, Associate Professor CentraleSupélec, GeePs
theodore.cherriere@centralesupelec.fr

Course Objectives

Skills and know-how

1. Define a model (reminder)

- Formalism: state, design variable, parameters, criteria
- Calculus of derivatives, adjoint method

2. Evaluating the condition of a non-linear model

- Fixed point method
- Newton method

3. Implement an unconstrained optimization approach

- Gradient descent
- Newton method

4. Implementing a constrained optimization approach

NEXT TIME

wooclap



app.wooclap.com/events/ESPSSYF

Exercises and problems

Link to exercises

https://tcherrie.github.io/optim_M2_PIE/lab/

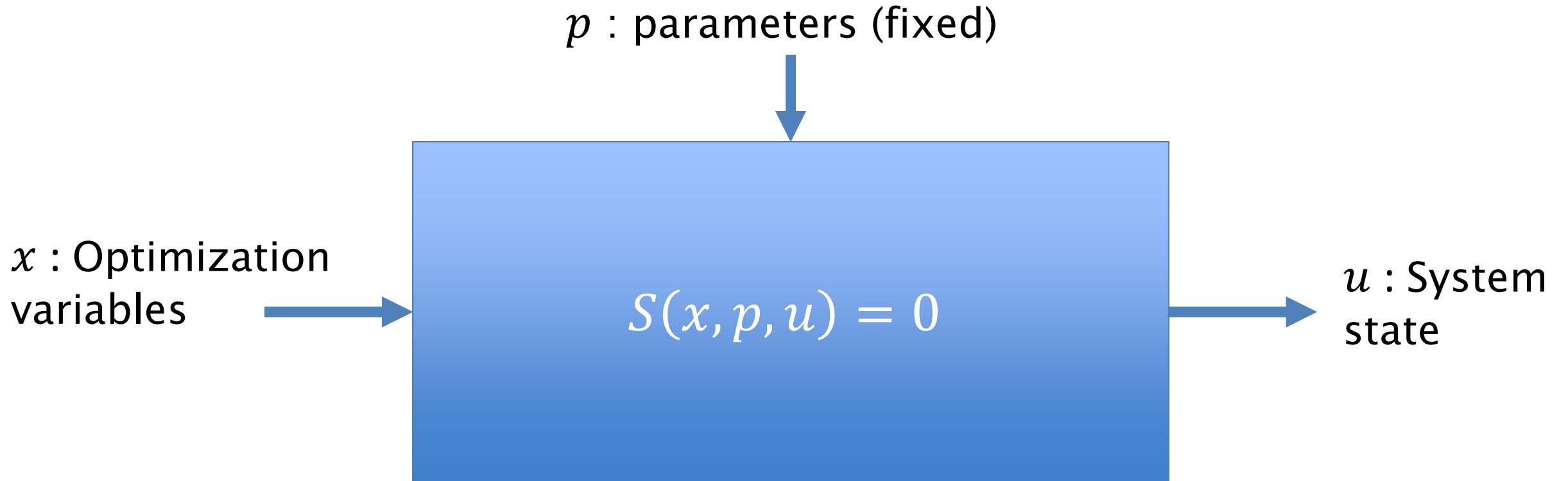


I) DEFINE A MODEL

Formalism, vocabulary, derivation

Formalism

What is meant by "model"?



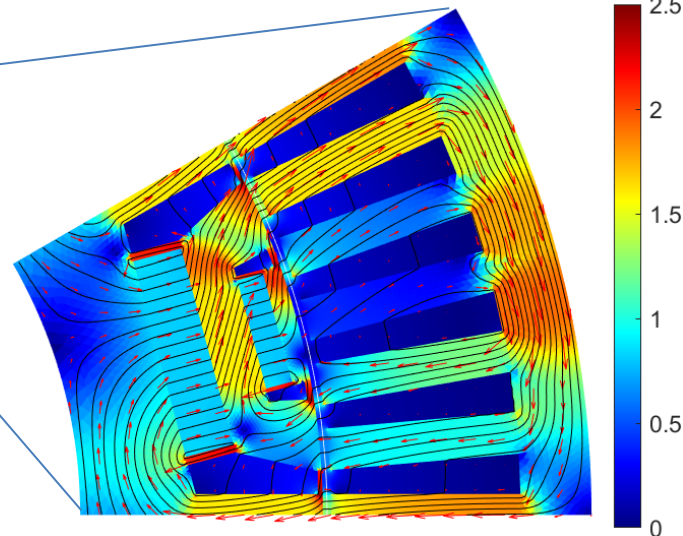
S : state equation of the system

Formalism

Example: Magnetostatic simulation

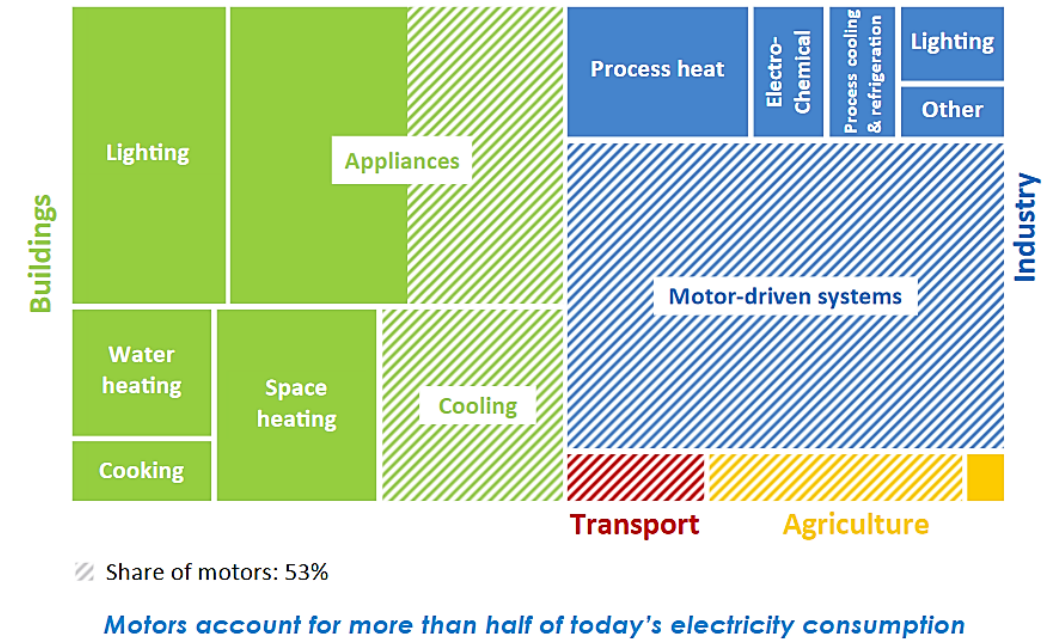
- *Electric motor*

Share in global electricity consumption: + de 50% !



Magnetic induction (T)

Figure 7.9 ▷ Global total final electricity consumption by end-uses, 2014



Source: IEA analysis.

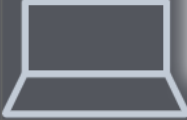
What are the parameters, design variables, equations of the model?

Allez sur wooclap.com et utilisez le code **ESPSSYF**



0 ?

Sort the physical quantities according to their role in the model.

Symbol	Type	Physical quantity
S	State equation	 
u	State variable	
p	Parameter (fixed)	
x	Optimization variable	

Cliquez sur l'écran projeté pour lancer la question

1 Maxwell's equations

2 Geometry

3 Flux density

4 Material constitutive law

5 Power supply



Formalism

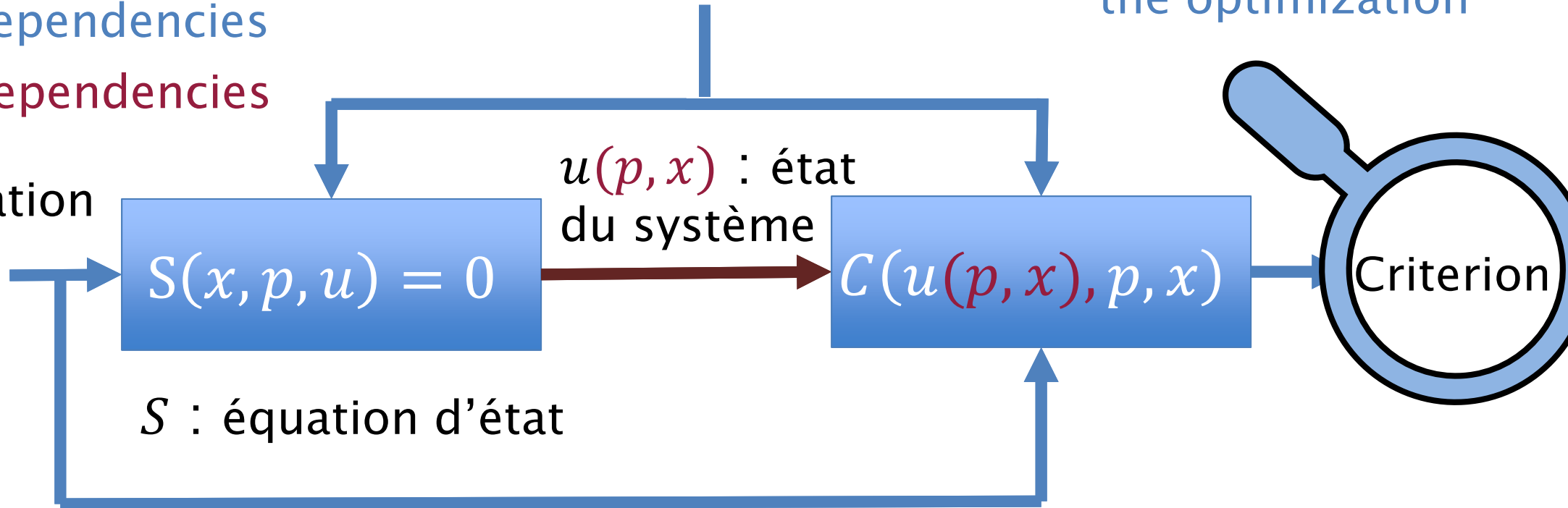
Criterion

- Explicit dependencies
- Implicit dependencies

x : optimization variables

p : fixed parameters

Quantity of interest in the optimization



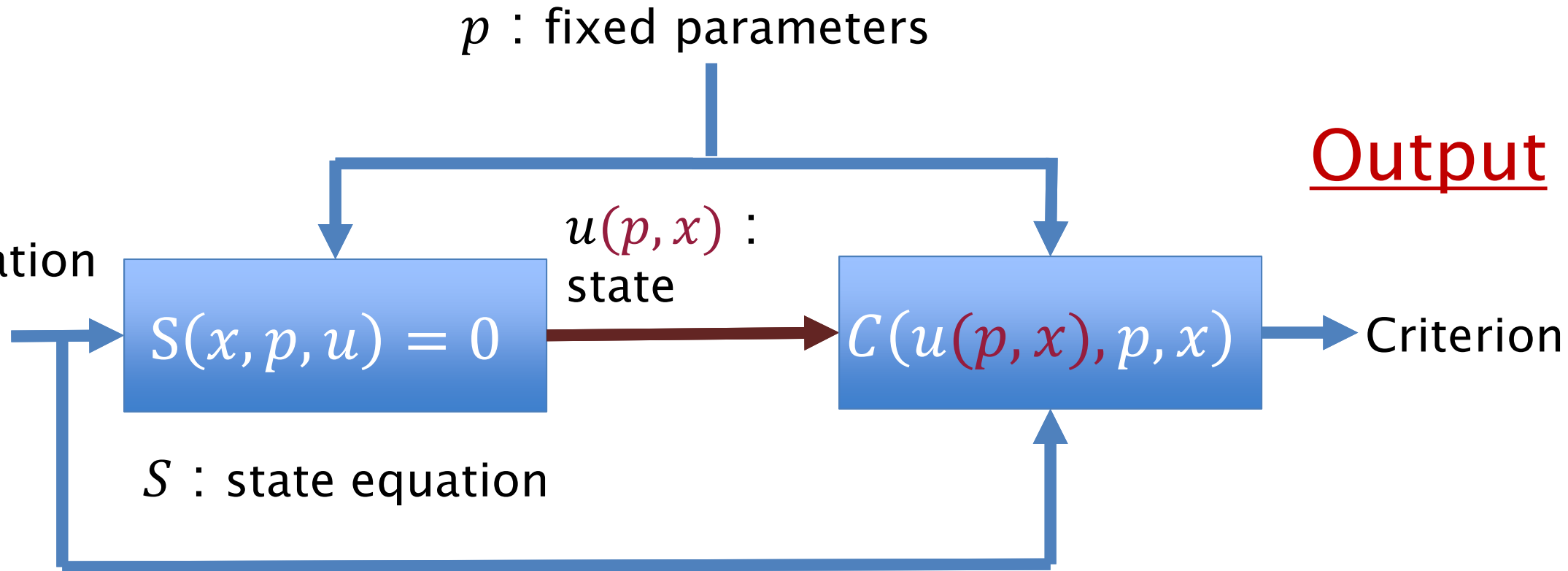
The quantities of interest generally depend **explicitly** on the state but **implicitly** on the design variables.

Direct problem

Definition

Input

x : optimization variables



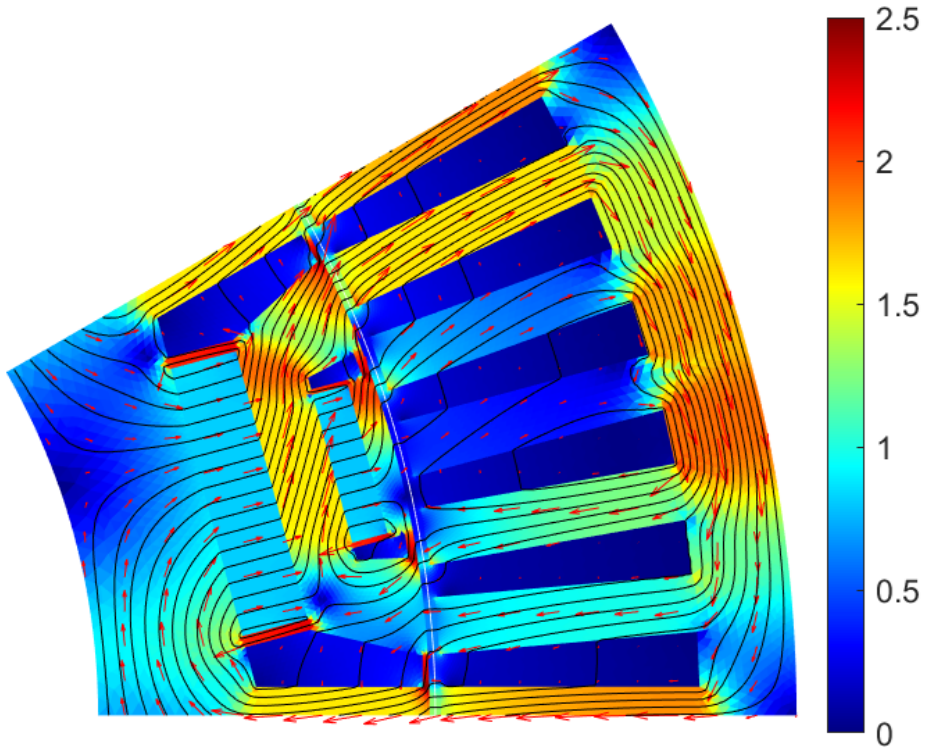
Output

Criterion

Solving a direct problem is "easy," but is rarely an objective in itself.

Direct problem

Example: Magnetic Torque Calculation



State $u \rightarrow \vec{B}$
(flux density)

Formula from the distribution of $\vec{B}$ in the air gap e (Arkkio, 1987) :

$$\Rightarrow T(\vec{B}) = \frac{L_z}{\mu_0 \Delta R} \int_e r B_r B_\theta \, dS$$

If we change the **power supply** (p) or the **geometry** (x), the state $\vec{B}$ (u) will change, and therefore $T(\vec{B}(x, p))$ as well.

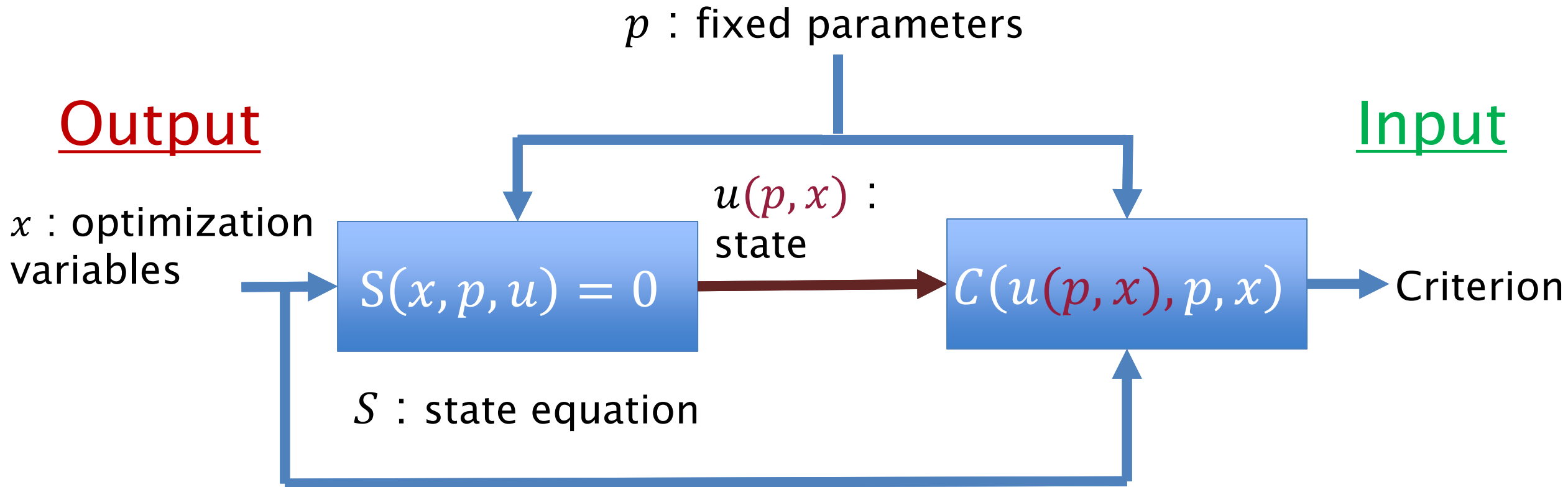
$\Rightarrow$ **Implicit dependence** via $S(x, p, u) = 0$

Direct problem

Definition

Output

Input



The customer is usually interested in the **inverse problem**: they look for the **optimization variables** associated with expected performance.

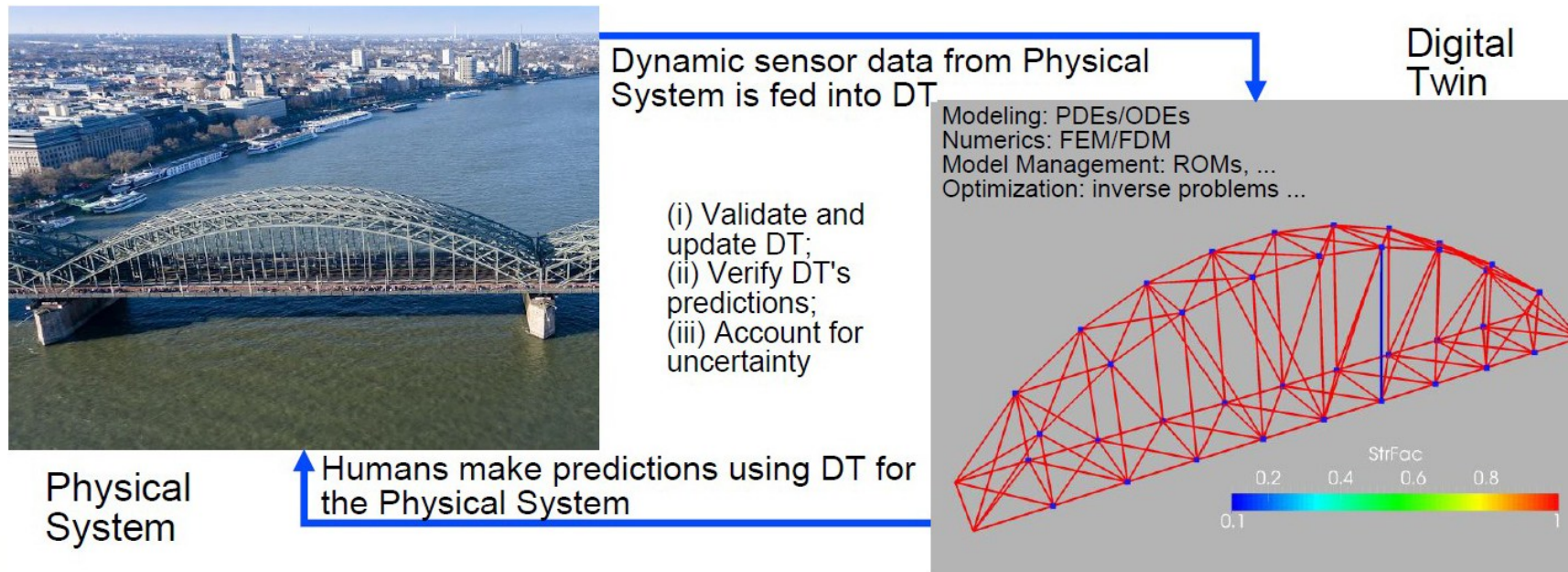
Inverse problem

Difficulties related to reverse problems

Solving the inverse problem is usually difficult:

- Not necessarily existence, uniqueness or stability of the solution x (ill-posed problem) → need for ***regularization, constraints***

Example: predictive maintenance by digital twin



Derivative calculation

In this course session, we will focus on methods based on derivatives for:

1. **solving** $S(x, p, u) = 0$ (reminder)
2. **optimizing** with respect to the design variable x ,

Note: x, p, u can be scalar, vector, or even infinite-dimensional.

Derivative calculation

Challenges

- First difficulty: dimension and nature of the quantities to derived

What you probably already know how to derivate:

- $f: \mathbb{R} \rightarrow \mathbb{R}$
- $f: \mathbb{R}^n \rightarrow \mathbb{R}$
- $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$



What you may not yet know how to derivate...

- $f: L^2(\Omega) \rightarrow \mathbb{R}$ (directional / Fréchet derivative)
- $f: \Omega \rightarrow \mathbb{R}$, with Ω a shape (shape, topological derivative)

In this course, we assume that we know how to derivate and we note the derivative of f

- f' if f depends only on one variable (possibly of infinite dimension)
 - $d_x f$ the **total** derivative with respect to x
 - $\partial_x f$ the partial derivative with respect to x
- if f depends on several variables

Derivative calculation

Challenges

- Second difficulty: **implicit dependencies** between variables.

Let us define $y = 2x$, $f_1 = x + y$, $f_2 = 3x$

Allez sur wooclap.com et utilisez le code **ESPSSYF**

0

Are f_1 and f_2 equal?

①

Yes

0%

0



②

No

0%

0



1) Are f_1 and f_2 equal?

It depends on what we are talking about!

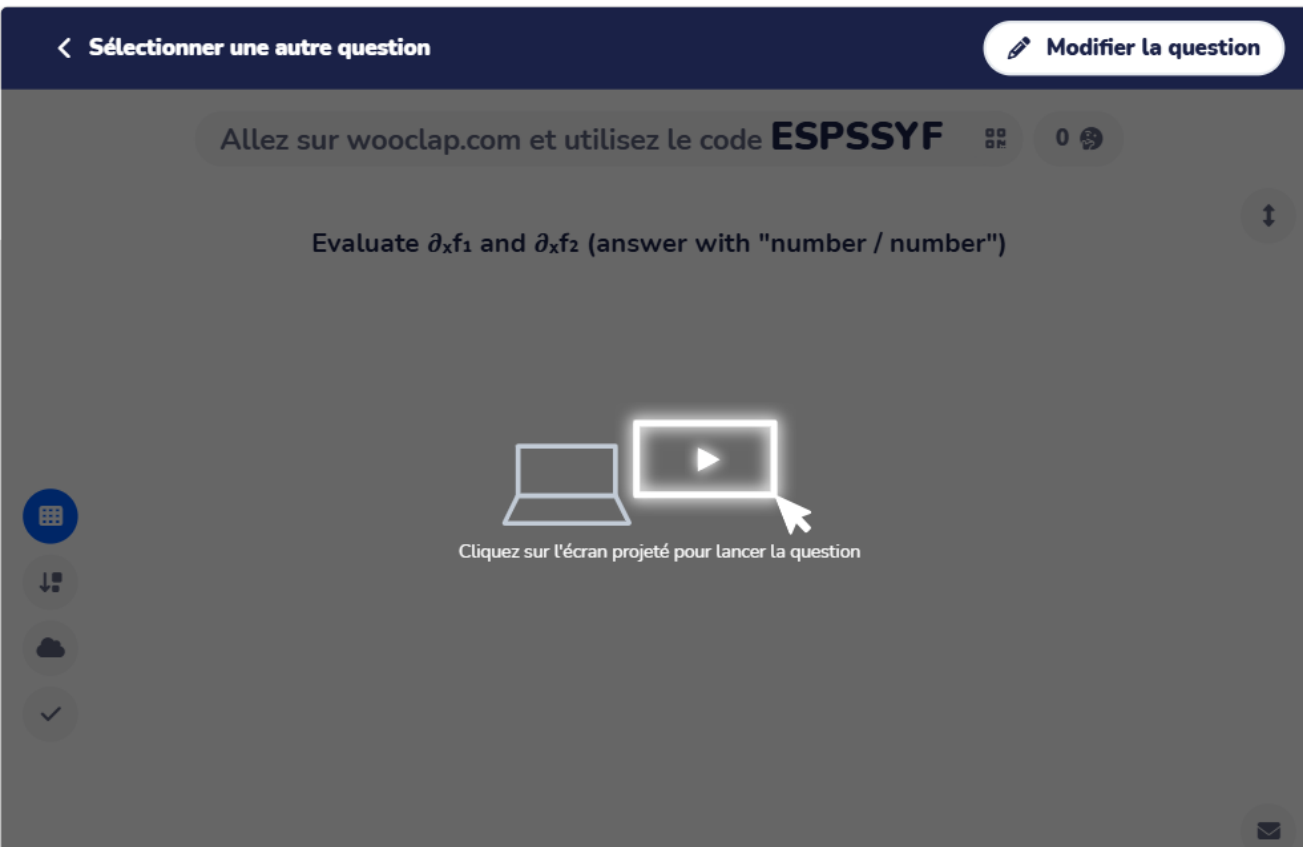
- **Quantities** : $f_1 = x + 2x = 3x = f_2$
- **Functions** : $f_1: x, y \mapsto x + y$
 $\neq f_2: x, y \mapsto 3x$

Derivative calculation

Challenges

- Second difficulty: **implicit dependencies** between variables.

Let us define $y = 2x$, $f_1 = x + y$, $f_2 = 3x$



2) Evaluate $\partial_x f_1$ and $\partial_x f_2$?

Partial derivative = "differentiate while holding the other variables constant"

Useful if you consider functions
 $\partial_x f_1 = 1$, $\partial_x f_2 = 3$

Derivative calculation

Challenges

- Second difficulty: **implicit dependencies** between variables.

Let us define $y = 2x$, $f_1 = x + y$, $f_2 = 3x$



3) Evaluate $d_x f_1$ and $d_x f_2$?

"Total" derivative: chain rule

$$d_x f(x, y) = \partial_x f + \partial_y f d_x y$$

Same quantity = same "total" derivatives

Derivative calculation

Finite difference estimation

If finite dimension design variable x :

Forward finite difference $\Delta_{x_i} f = f(u(x_1, x_2, \dots, x_i + h, \dots, x_n)) - f(u(x_1, x_2, \dots, x_i, \dots, x_n))$

Centered finite difference $\delta_{x_i} f = f\left(u\left(x_1, x_2, \dots, x_i + \frac{h}{2}, \dots, x_n\right)\right) - f\left(u\left(x_1, x_2, \dots, x_i - \frac{h}{2}, \dots, x_n\right)\right)$

Forward FD	Centered FD
$d_{x_i} f - \frac{\Delta_{x_i} f}{h} = O(h)$	$d_{x_i} f - \frac{\delta_{x_i} f}{h} = O(h^2)$

[Jupyter Notebook](#)

Derivative calculation

Complex step

If a finite-dimensional design variable x and f is analytic:

Complex step $\underline{\Delta_{x_i} f} = f(u(x_1, x_2, \dots, x_i + ih, \dots, x_n)) - f(u(x_1, x_2, \dots, x_i, \dots, x_n))$

Forward FD	Centered FD	Complex step (f is analytic)
$d_{x_i} f - \frac{\Delta_{x_i} f}{h} = O(h)$	$d_{x_i} f - \frac{\delta_{x_i} f}{h} = O(h^2)$	$d_{x_i} f - \frac{\text{Im}(\underline{\Delta_{x_i} f})}{h} = O(h^2)$ No floating point computation errors

Simple to implement, robust, works even if $f(u(x))$ black box...

... but must be repeated for each component of x !! **SLOW**



Derivative evaluation

Exercise

- Exercise A on the website

https://tcherrie.github.io/optim_M2_PIE/lab/

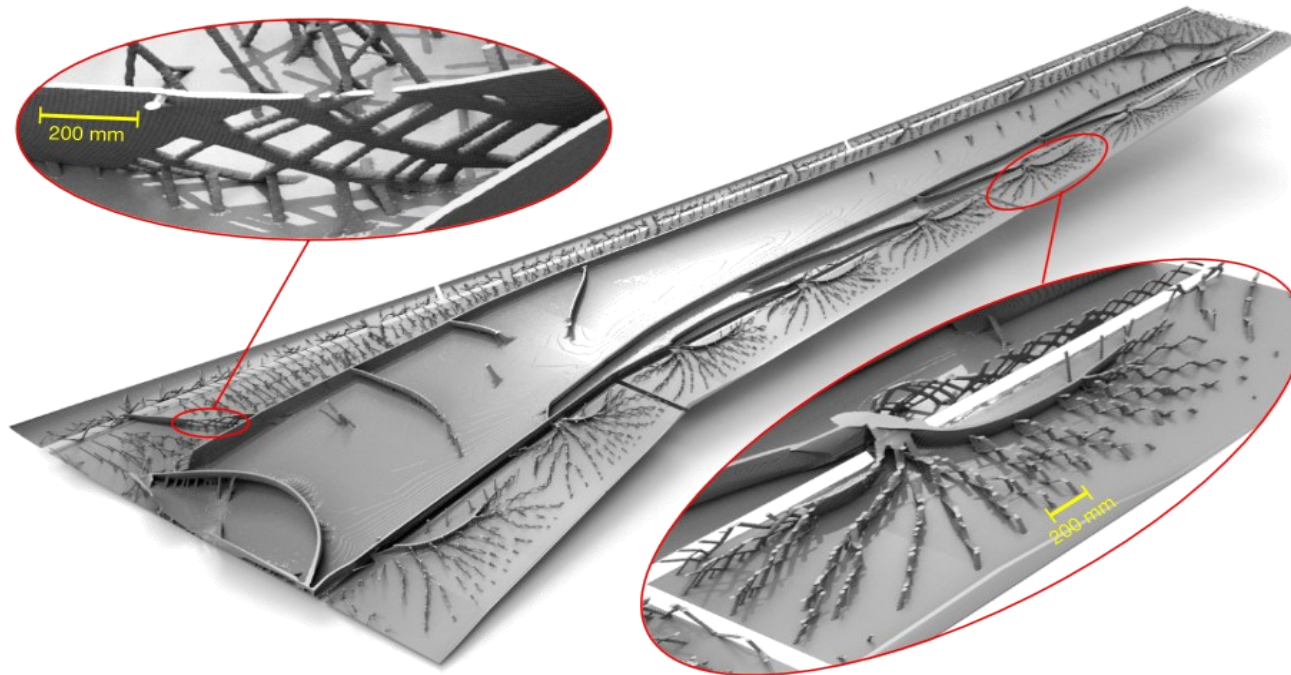
Derivative calculation

Large scale problems

Giga-voxel computational morphogenesis for structural design

| NATURE | VOL 550 | 5 OCTOBER 2017

Niels Aage^{1,2}, Erik Andreassen¹, Boyan S. Lazarov¹ & Ole Sigmund¹



element i . The sensitivities of the objective function are determined efficiently without additional analysis using the **adjoint method**, and the filtering operation is included by the **chain rule**²². The optimization problem is solved using a nested approach and an optimality criteria algorithm^{2,23} (simple MATLAB code available

Figure 2 | Morphogenesis evolution, 3D printing and comparison to the hornbill bird beak. a, Steps of the morphogenesis process for the full-wing

Derivative calculation

Adjoint method (formal principle)

We aim to compute $d_{x_i}f$. **Chain rule:**

$$d_{x_i}f = \partial_{x_i}f + \partial_u f d_{x_i}u \quad (1)$$

Step 1: derive the equation of state to make appear $d_{x_i}u$ using the **chain rule**:

$$\begin{aligned} d_{x_i}S &= \partial_{x_i}S + \partial_u S d_{x_i}u = 0 \\ \Rightarrow \underbrace{\partial_u S d_{x_i}u}_{\text{blue bracket}} &= -\partial_{x_i}S \quad (2) \end{aligned}$$

Step 2: introduce $\partial_u S$ inside (1)

$$\partial_u f d_{x_i}u = \partial_u f \underbrace{\partial_u S^{-1} \partial_u S d_{x_i}u}_{\text{blue bracket}} = -\partial_u f \partial_u S^{-1} \partial_{x_i}S$$

Step 3: solve the **adjoint system** (always linear), independent of x_i

$$p \partial_u S = -\partial_u f \Leftrightarrow \partial_u S^* p = -\partial_u f$$

Finally,

$$\boxed{d_{x_i}f = \partial_{x_i}f + p \partial_{x_i}S}$$

Derivative calculation

Adjoint method

In summary:

1. Calculation of the **physical state** u (not necessarily linear = **long**); we solve

$$S(u, x) = 0$$

2. Calculation of the **adjoint state** p (always linear, sometimes self-adjoint $\Rightarrow$ **fast**); we solve

$$\partial_u S^* p = -\partial_u f$$

3. The following formula is applied for each component x_i (**rapide**):

$$d_{x_i} f = \partial_{x_i} f + p \partial_{x_i} S$$

Derivative calculation

Adjoint method

Example of a high-dimensional problem
Topological optimization of a transformer

[Jupyter Notebook](#)

Adjoint method

Exercise

- Exercise B on the website

https://tcherrie.github.io/optim_M2_PIE/lab/

Derivative calculation

Symbolic and automatic derivation

Other approaches:

- **Symbolic derivative** (explicit construction of an expression on which the usual rules of differentiation are applied)
- **Automatic derivative:** derivatives are computed numerically and recursively, based on the chain rule. A distinction is made between:
 - Forward accumulation: more efficient when the dimension of the arrival space is large (rare)
 - Backward accumulation: more efficient when the size of the starting space is large (e.g. neural networks)
 - *Python libraries:* JAX, PyTorch, etc.

2) SOLVE THE STATE EQUATION

Fixed Point Method, Newton's Method

Fixed point algorithm

Objective

We now want to find u^* which solves the equation of state

$$S(u^*, x, p) = 0$$

Since x and p are fixed, we simply write

$$S(u^*) = 0$$

We are therefore looking for an algorithm that generates a sequence (u_k) , such as

$$\lim_{k \rightarrow \infty} u_k = u^*$$

Fixed point algorithm

Principle

Consider $F: u \mapsto u + A(u) S(u)$, with A any invertible operator. Solve $S(u^*) = 0$ is equivalent to finding a fixed point of F :

$$u^* = F(u^*)$$

Let's apply the simplest possible algorithm:

$$u_{k+1} = F(u_k)$$

And let's study its convergence.

Fixed point algorithm

q - convergence

Suppose that u^* exists and that F is differentiable; we consider u_k close to u^* . We apply Taylor expansion:

$$\frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} = \frac{\|F(u_k) - u^*\|}{\|u_k - u^*\|} = \frac{\|F(u^*) + F'(u^*)(u_k - u^*) + O(u_k - u^*)^2 - u^*\|}{\|u_k - u^*\|}$$

$$\Rightarrow \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} \rightarrow \|F'(u^*)\|$$

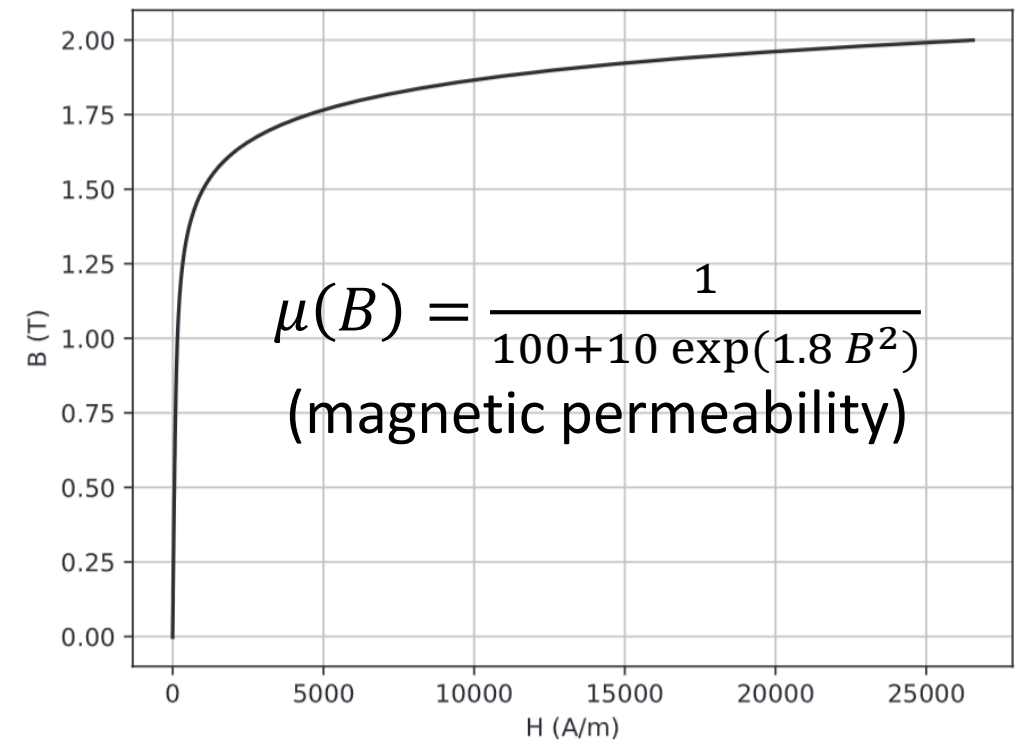
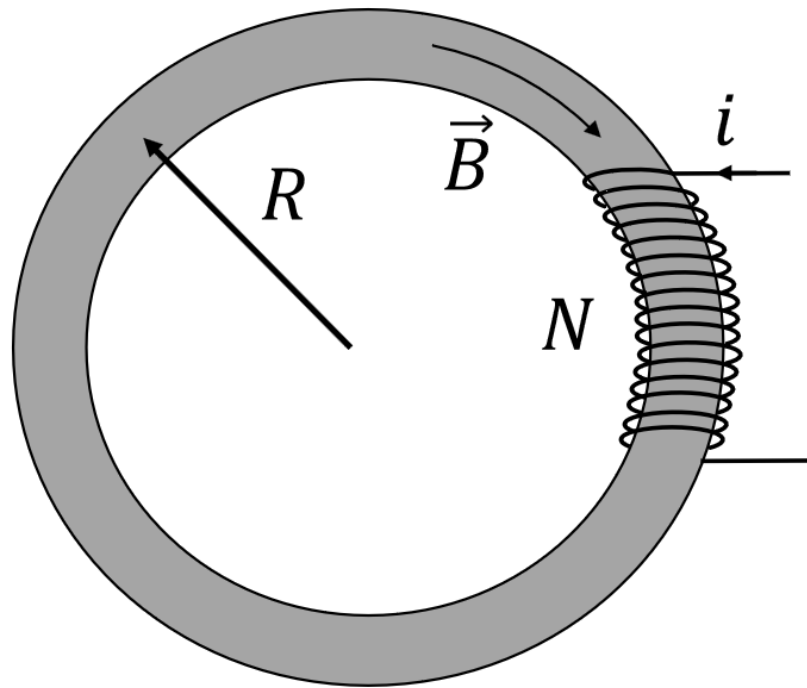
$\ F'(u^*)\ = 0$	$\ F'(u^*)\ \in]0,1[$	$\ F'(u^*)\ > 1$
Superlinear convergence	Linear convergence	No convergence
Fast		Unstable

Fixed point algorithm

Numerical example

Magnetic induction in a ferromagnetic core

- $N = 100$
- $i = 5 \text{ A}$
- $R = 5 \text{ cm}$



[Jupyter notebook](#)

Convergence acceleration

Aitken Δ^2 method

- Assume $\|F'(u^*)\| \in]0,1[\Rightarrow F'(u^*) = \tau \in]-1,0[\cup]0,1[$. If u is scalar, we have

Linear convergence $\Rightarrow (u_k) \rightarrow u^* + \tau^k$

$$\Rightarrow \frac{u_{k+1} - u^*}{u_k - u^*} \rightarrow \tau \Rightarrow u^* \rightarrow u_k - \frac{u_{k+1} - u_k}{\tau - 1}$$

But τ is unknown. How can it be estimated?

$$\lim_{k \rightarrow \infty} \frac{u_{k+1} - u_k}{u_k - u_{k-1}} = \tau \Rightarrow \tau \rightarrow \frac{u_{k+2} - u_{k+1}}{u_{k+1} - u_k}$$

A more accurate estimate of the limit is derived from

$$u^* \rightarrow u_k - \frac{(u_{k+1} - u_k)^2}{(u_{k+2} - u_{k+1}) - (u_{k+1} - u_k)} = u_k - \frac{(\Delta u_k)^2}{\Delta^2 u_k}$$

The next suite therefore converges faster to the solution:

$$(\tilde{u}_k) = u_{k-2} - \frac{(u_{k-1} - u_{k-2})^2}{(u_k - u_{k-1}) - (u_{k-1} - u_{k-2})}$$

Convergence acceleration

Vector Aitken Δ^2 method

- Each component of the vector converges (at least) linearly
 $\Rightarrow$ we can apply the Aitken method component by component...
... But we don't take advantage of any couplings between the components.
- To do this, we would have to use the inverse of a vector(?)
 - There is no single satisfactory way to proceed.
 - Recommendation: $U^{-1} = \frac{U}{\|U\|^2}$ (pseudo-inverse)
- Generalization: ϵ – algorithm

[P. Wynn, « On a device for computing the $e_m(S_n)$ transformation », [MTAC](#), 10 (1956), p. 91-96]

Convergence acceleration

Choice of A

Fixed point algorithm: $u_{k+1} = F(u_k)$ with $F: u \mapsto u + A(u) S(u)$

$$\frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} = \frac{\|F(u_k) - u^*\|}{\|u_k - u^*\|} = \frac{\|F(u^*) + F'(u^*)(u_k - u^*) + O(u_k - u^*)^2 - u^*\|}{\|u_k - u^*\|}$$

$$\Rightarrow \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} \rightarrow \|F'(u^*)\|$$

$\ F'(u^*)\ = 0$	$\ F'(u^*)\ \in]0,1[$	$\ F'(u^*)\ > 1$
Superlinear convergence	Linear convergence	No convergence

Fast

Choice of A can speed up or stabilize the algorithm

Unstable

Newton method

Principle

- We want to solve $S(u) = 0$, we know how to solve by fixed point $F(u) = u$ with $F: u \mapsto u + A(u) S(u)$; let's write the derivative to the solution u^*

$$F'(u^*) = Id + \underbrace{A'(u^*) S(u^*)}_0 + A(u^*) S'(u^*) = Id + A(u^*) S'(u^*)$$

- Necessary condition of superlinear q-convergence :

$$\|F'(u^*)\| = 0 \Rightarrow A(u^*) = -(S'(u^*))^{-1}$$



Impossible if
 $S'(u^*) = 0$!

- We deduce Newton (Raphson) method:

$$u_{k+1} = u_k - (S'(u_k))^{-1} S(u_k)$$

Newton method

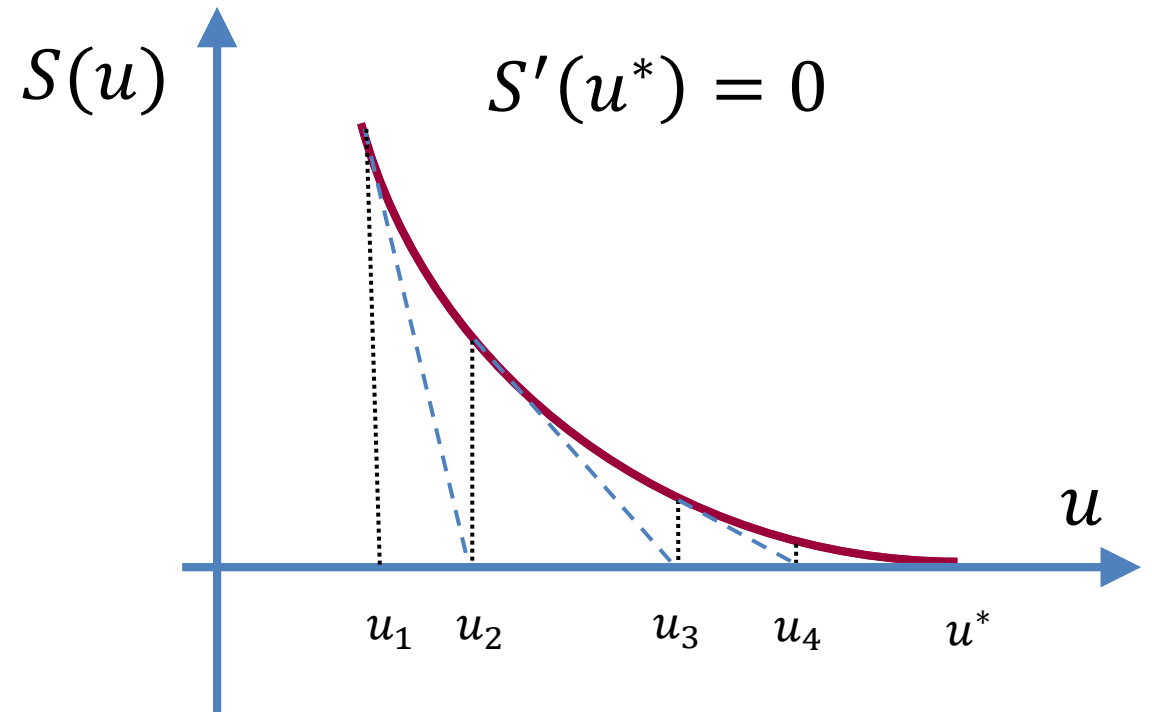
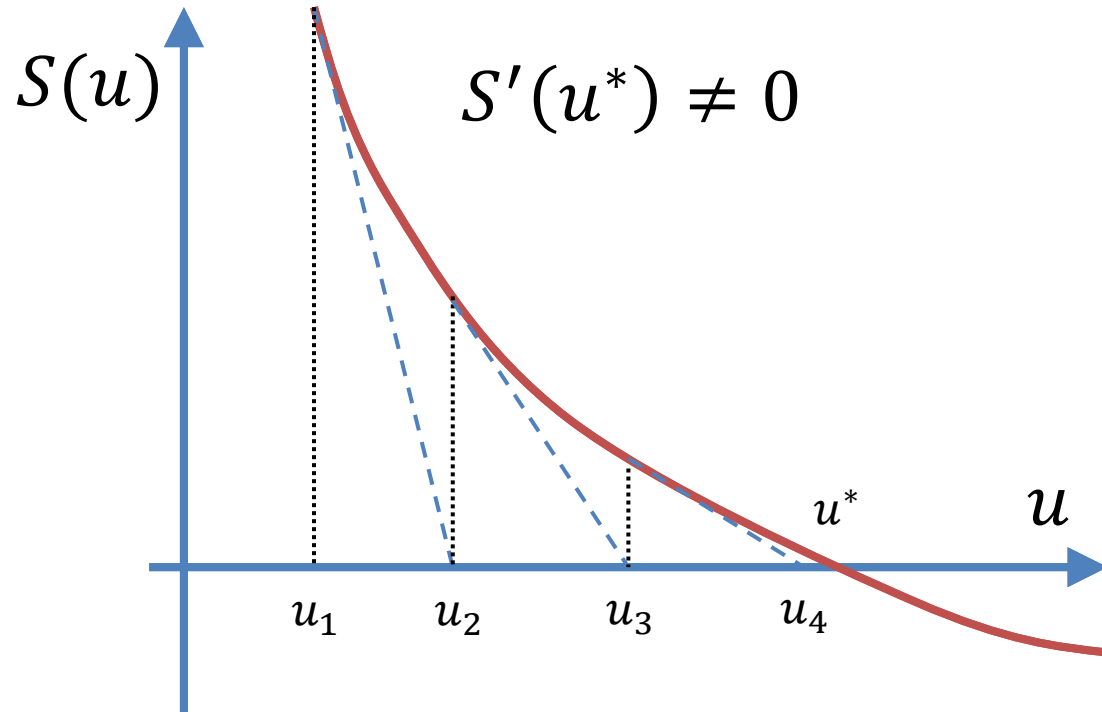
Limitation

Question: In the scalar case, can we still apply Newton's method if $S'(u^*) = 0$?

The screenshot shows a presentation slide on the Wooclap platform. At the top, there is a navigation bar with a back arrow and the text "Sélectionner une autre question" on the left, and a "Modifier la question" button on the right. Below this, a banner instructs users to go to wooclap.com and use the code **ESPSSYF**. The main content area displays the question: "In the scalar case, can we still apply Newton method if $S'(u^*)=0$?". Below the question is a video player interface with a play button icon and a laptop icon. A white box highlights the play button, and a mouse cursor points to it. Below the video player, the text "Cliquez sur l'écran projeté pour lancer la question" is displayed. On the left side of the slide, there is a blue circular icon with a list symbol and a circled number "1". At the bottom right, there is a progress indicator showing "0%" and "0" people. The Wooclap logo is in the bottom left corner, and the "uni PARI" logo is in the bottom right corner.

Newton method

Graphical illustration



- Newton's method can be applied even if $S'(u^*) = 0$, but we lose the **superlinear** convergence.

Example (in exercise): $S(u) = (u - u^*)^m \Rightarrow \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|} \rightarrow 1 - \frac{1}{m}$

Newton method

q-convergence

Assume u^* exists, S is differentiable and $S'(u^*) \neq 0$; consider u_k *near* u^* .
Taylor expansion on $F: u \mapsto u - (S'(u))^{-1} S(u)$:

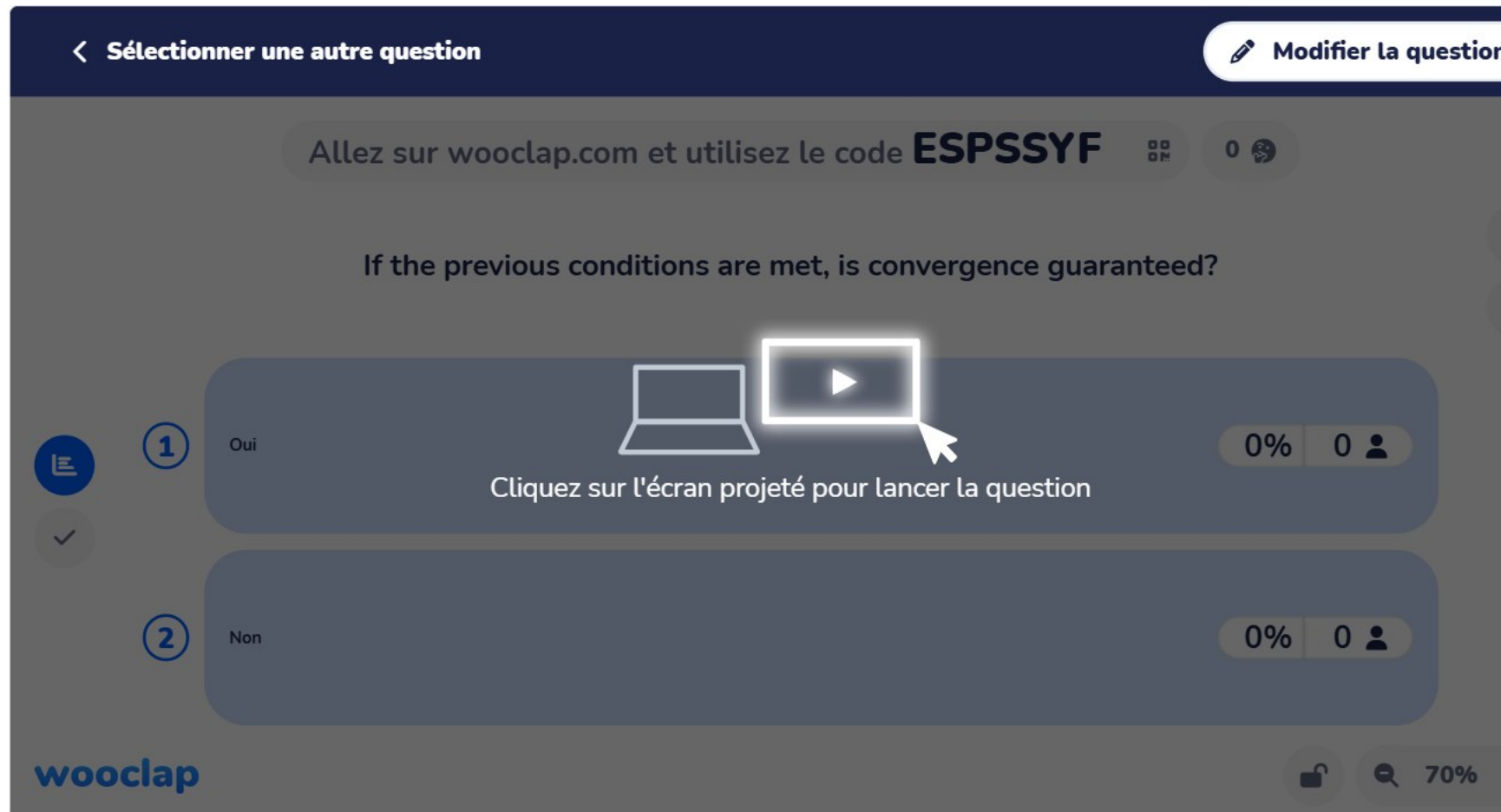
$$\begin{aligned} \exists \epsilon < \infty, \quad \frac{\|u_{k+1} - u^*\|}{\|u_k - u^*\|^2} &\leq \frac{\|F(u^*) + F'(u^*)(u_k - u^*) - u^*\| + \epsilon \|u_k - u^*\|^2}{\|u_k - u^*\|^2} \\ &= \frac{\left\| \left(Id - [(S'(u^*))^{-1}]' S(u^*) - (S'(u^*))^{-1} S'(u^*) \right) (u_k - u^*) \right\| + \epsilon \|u_k - u^*\|^2}{\|u_k - u^*\|^2} = \epsilon \end{aligned}$$

The q-convergence of Newton's method is **quadratic** (very fast!)

Newton method

Convergence

The previous conditions being met, is the convergence ensured?



Newton method

Convergence

- LOCAL convergence is assured, but certainly not global!

[Jupyter notebook](#)

Newton method

Line search

- To avoid divergence, a step adaptation (**line search**, in a fixed direction) can be introduced:

$$u_{k+1} = u_k - \alpha \left(S'(u_k) \right)^{-1} S(u_k), \quad \alpha \in (0,1]$$

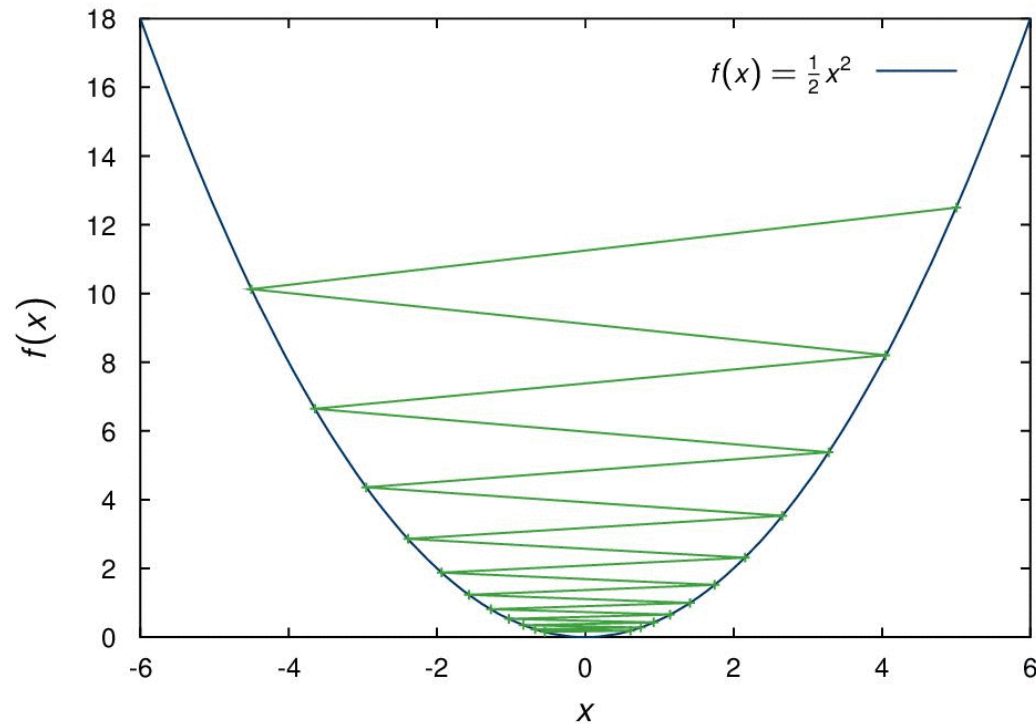
- Consider $f_k = S(u_k)^2$, we expect $f_{k+1} < f_k$ at each iteration
 $\Rightarrow S(u)$ goes to 0
- Linear search makes sense if $d_k = -\left(S'(u_k) \right)^{-1} S(u_k)$ is a **descent direction** for f_k . We can verify this:

$$d_k \cdot f'_k = -2S(u_k)S'(u_k) \left(S'(u_k) \right)^{-1} S(u_k) = -2S(u_k)^2 < 0 \text{ if } S(u_k) \neq 0$$

Newton method

Armijo's condition

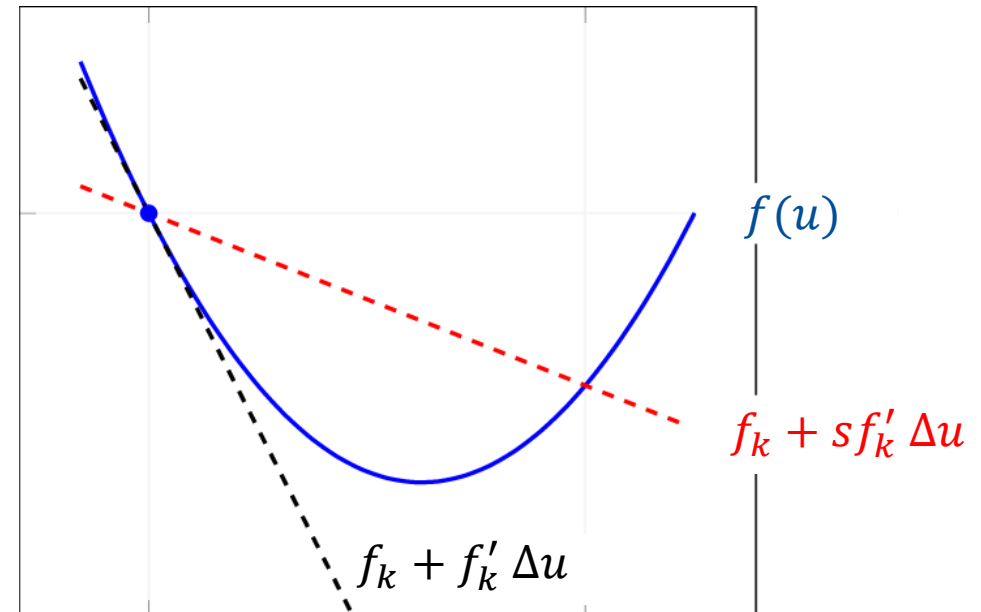
We expect to have, at least, $f_{k+1} \leq f_k$, but it may stagnates.



<https://medium.com/codex/gradient-descent-and-back-tracking-line-search-d8bd120bd625>

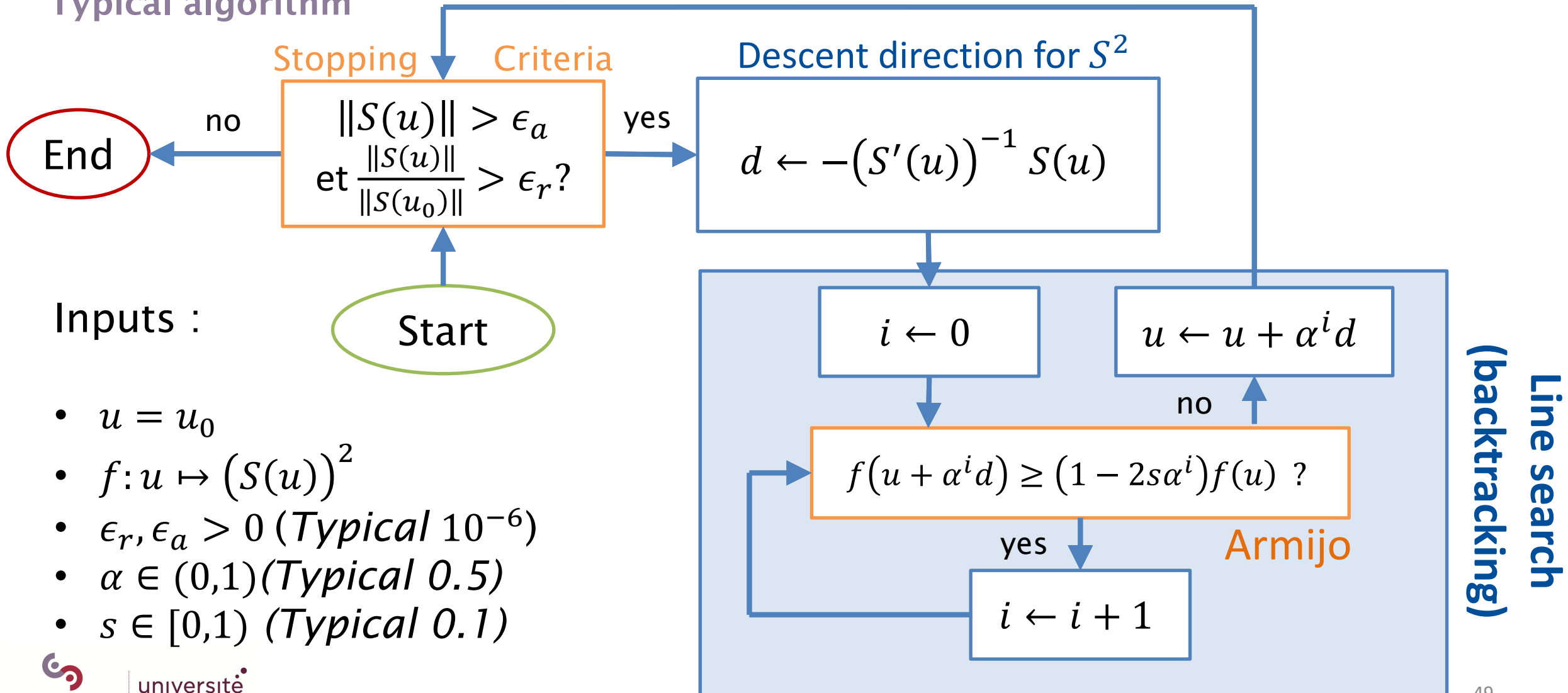
- You can have oscillations if the step is too big. **Armijo's condition** prevents this, with $0 \leq s < 1$:

$$f_{k+1} < f_k + s f'_k (u_{k+1} - u_k)$$



Newton method

Typical algorithm



Newton method

Application

[Jupyter notebook](#)

Newton method

Exercise

3) UNCONSTRAINED OPTIMIZATION

Gradient descent, Newton's method (bis)

Reminders

Direct problem

So far, we have seen how

- calculate the state u of a system (Fixed Point, Newton)
- calculate a criterion $\mathcal{C}(u(x))$
- calculate its derivative $d_x \mathcal{C}$ (finite differences, adjoint)

⇒ Direct problem solving

Generally, these problems are well posed (existence, uniqueness and stability of u)

Optimization problem

General case

- Optimization is closer to solving the opposite problem.
- It is a question of finding an **admissible solution** $x^* \in \Omega$, **minimizing a criterion** while **respecting constraints**.

A **single-objective optimization** problem is written in standard form

Find	$x^* = \arg \min_{x \in \Omega} F(x, p)$	Objective function (cost)
Subject to:	$h(x, p) = 0$	Equality constraint
	$g(x, p) \leq 0$	Inequality constraint

Initially, we limit ourselves to **non-constrained problems**.

Gradient descent – fixed step

Principle

- **Property** : $-d_x F$ is a **descent direction** of F

Indeed, $f(x - \alpha d_x F(x)) = f(x) - \alpha (d_x F(x))^2 + o(\alpha) \leq f(x)$ for $\alpha > 0$ small enough.

- We deduce the **fixed-step gradient descent** algorithm:

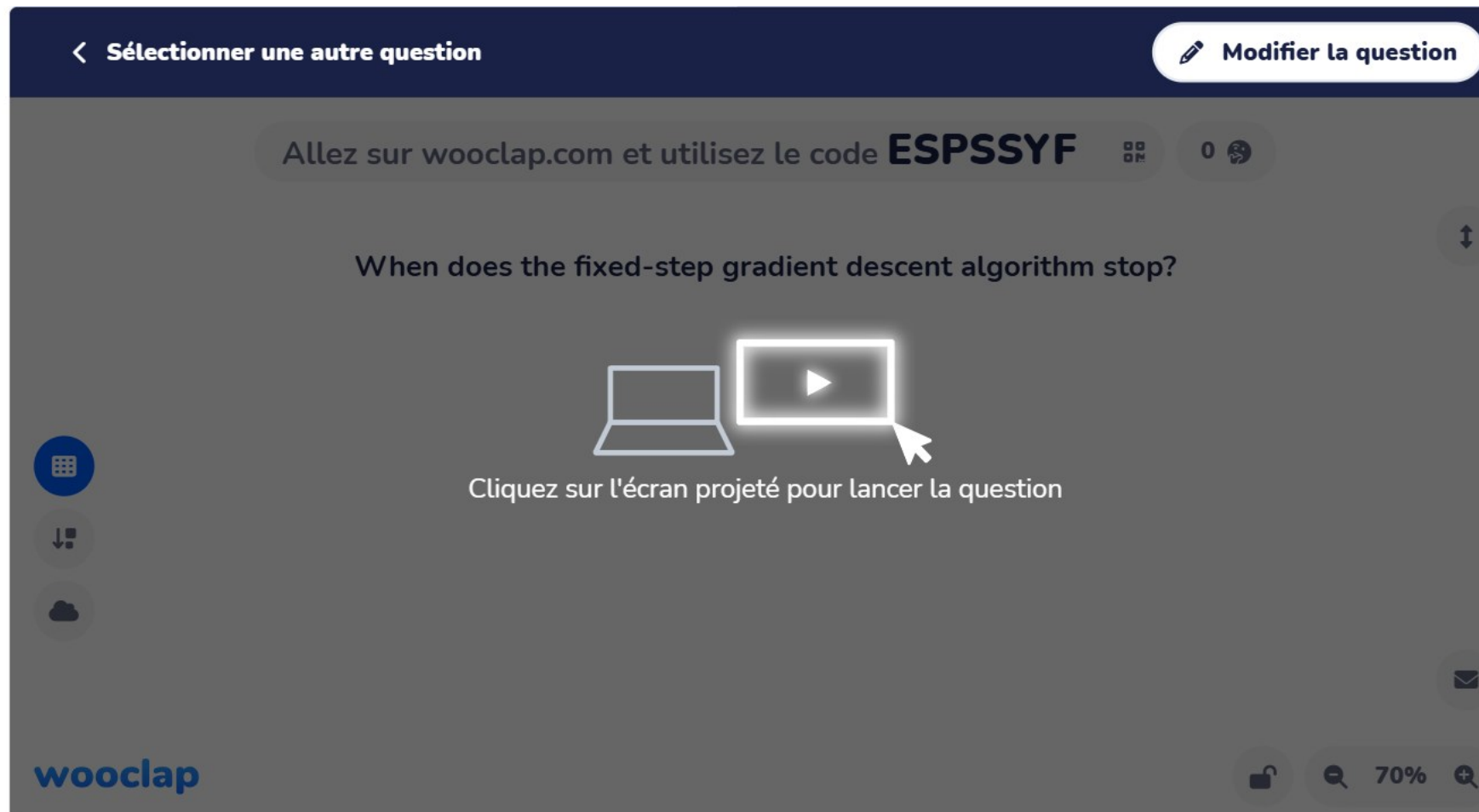
$$x_{k+1} = x_k - \alpha d_x F(x_k)$$

With $\alpha > 0$ the **step size**.

Gradient descent – fixed step

Stopping conditions

When does the algorithm stop?



Gradient descent – fixed step

Stopping conditions

Necessary optimality condition : $\|d_x F\| = 0$ (sufficient for a locally convex function)

$\Rightarrow$ Stop criterion based on the derivative norm: $\|d_x F\| < \epsilon_1$

Variants :

- Infinity norm: $\max_{i=1,\dots,n} \frac{\partial f(x)}{\partial x_i} < \epsilon_2$
- Stagnation of the objective function: $\left| \frac{f(x_{k+1}) - f(x_k)}{f(x_k)} \right| < \epsilon_3$
- Stagnation of design variable : $\frac{\|x_{k+1} - x_k\|}{\|x_k\|} < \epsilon_4$

...

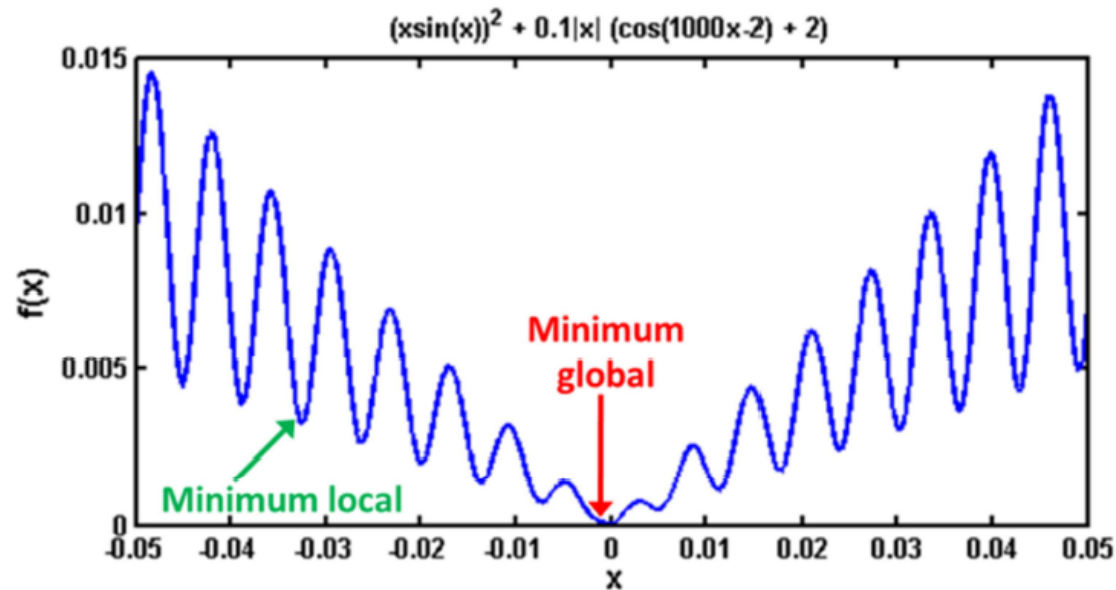
Note: always set a maximum number of iterations!

Local optima

Problem

The algorithm stops when $d_x F = 0$ (**critical point, optimality NC**)

- Local maximum (unstable), saddle point (usually not a problem)
- Otherwise, it is a **local minimum**...



Possible solutions:

- Multi-start
- *Global optimization (lecture P. Dessante).*

In the following, we will consider strongly convex functions
$$: F(tx + (1-t)y) > tF(x) + (1-t)F(y) \quad (\exists! \text{ minimum}).$$

Local optima

Examples

Examples of local optima: (free) topologies of electric machines maximizing the average torque at fixed supply.



Results obtained after random initializations – (multi-start)

Gradient descent – fixed step

Convergence

- The algorithm converges if the step is small enough.
- But the smaller it is, the more iterations it takes

Numerical example

Livescript matlab

Gradient descent – optimal step

Principle

We solve at each iteration:

$$\alpha^* = \arg \min_{\alpha} f(x_k - \alpha d_x F(x_k)) \quad [\text{Line search}]$$

$$x_{k+1} = x_k - \alpha^* d_x F(x_k) \quad [\text{Update}]$$

- **Automatic step adjustment:** convergence guaranteed if the minimum exists and is unique.
- Not necessarily efficient (finding the optimal step can be slow without being necessary)
- **Backtracking** by Armijo's method is generally more efficient.

Illustration numérique
[Livescript matlab](#)

Preconditioned gradient descent

Principle

- **Preconditioning** with a positive operator P .

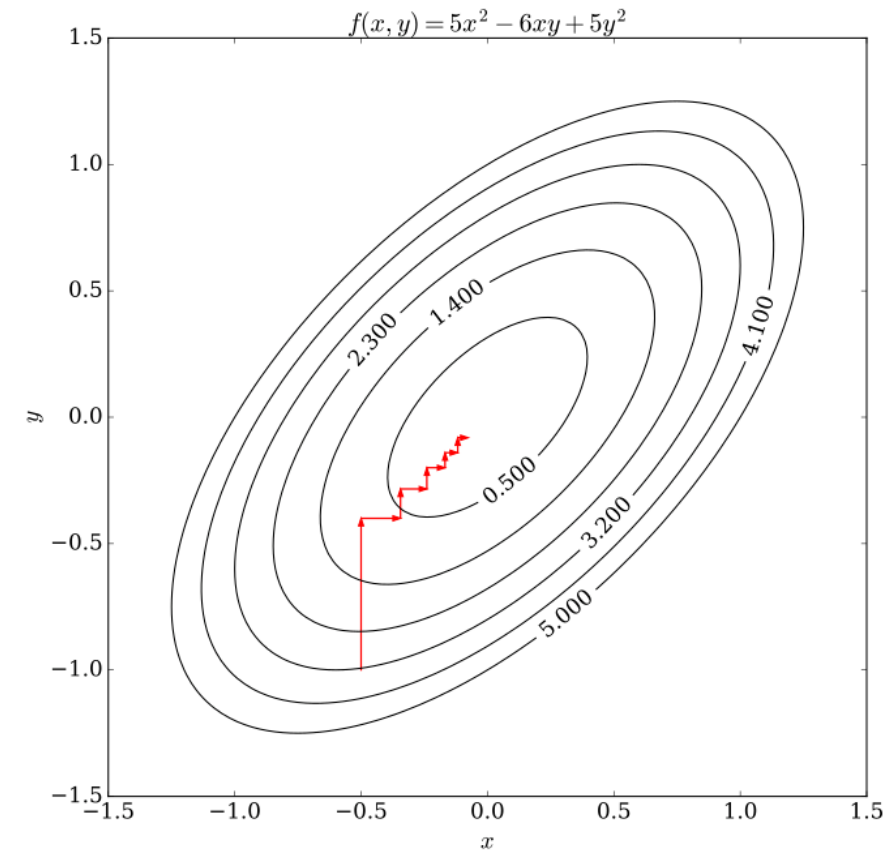
$$d_k = -P d_x F(x_k) \quad [\text{descent direction}]$$

$$\alpha^* = \arg \min_{\alpha} f(x_k + \alpha d_k) \quad [\text{line search}]$$

$$x_{k+1} = x_k + \alpha^* d_k \quad [\text{update}]$$

Example: "coordinate descent«
(*numerical example*), or... Newton!

[Livescript matlab](#)



Newton method

Principle

- We have already seen the principle of Newton's method for finding the zero of a function.
- This method can therefore be applied to $d_x F$ for the search of the **critical point** of F (zero of its derivative)!

$$x_{k+1} = x_k - \alpha (d_{xx}F)^{-1} d_x F$$

- Only works if $d_{xx}F$ positive definite (characterization of a local minimum, in relation to a saddle point or a local maximum)
- Exact if the function is (locally) quadratic!
- We can decompose into quadratic sub-problems: SQP idea

Newton method

Application

- Rosenbrock function

[Livescript matlab](#)

Quasi-Newton

Principle

- The calculation of the first derivative of F ("gradient" $\rightarrow$ vector) is not always easy (adjoint method)
- The calculation of the second derivative ("Hessian" $\rightarrow$ matrix) is even less so! If x is large, memory issues may occur.
- Secant method, BFGS: the information from the previous steps is **reused** to estimate the Hessian matrix
- We can still achieve a **superlinear convergence**, even without knowing the exact value of the Hessian one! But we have to make sure that it is **positively definite symmetric**.

4) CONSTRAINED OPTIMIZATION

Reformulations, algorithmes

Optimization problem

General case

- Optimization is closer to solving the opposite problem.
- It is a question of finding an **admissible solution** $x^* \in \Omega$, **minimizing a criterion** while **respecting constraints**.

A **single-objective optimization** problem is written in standard form

Find	$x^* = \arg \min_{x \in \Omega} F(x, p)$	Objective function (cost)
Subject to:	$h(x, p) = 0$	Equality constraint
	$g(x, p) \leq 0$	Inequality constraint

Projected gradient descent

Principle

- Some constraints are easy to take into account.
- We can therefore project the variables onto the admissible space Ω .

$$x_{k+1} = P_{\Omega}(x_k - \alpha F'(x))$$

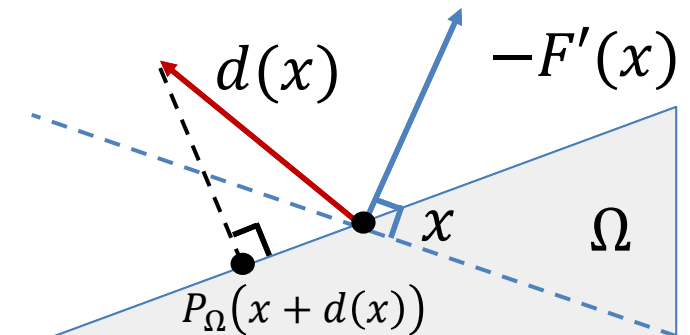
With

- α the step size
- $F'(x)$ the derivative of F
- P_{Ω} the orthogonal projection on Ω (**not always easy**). In the case of "box" constraints

$x > x_{min}$ ou $x < x_{max}$:

$$P_{\Omega}(x) = \max(x_{min}, \min(x_{max}, x))$$

Be careful: any other direction of descent is not necessarily the same after projection



- **Stopping criterion:** need to project the gradient as well!

Penalty methods

Principle

- We can transform the constrained problem into an **unconstrained** problem (we can then apply the descent methods).
- The idea is to add to the objective function to be minimized **important penalty terms** when constraints are violated.

$$F_p(x) = F(x) + P_h(x) + P_g(x)$$

With

- $h(x) = 0 \Rightarrow P_h(x) = 0,$ $h(x) \neq 0 \Rightarrow P_h(x) \gg 1$
- $g(x) \leq 0 \Rightarrow P_g(x) = 0,$ $g(x) > 0 \Rightarrow P_h(x) \gg 1$

Penalty methods

Principe

Any ideas of penalization functions to use? (equality constraint)



Penalty methods

Principe

Any ideas of penalization functions to use? (inequality constraint)



Penalty methods

Exterior

Example of **exterior** penalty ($\mathcal{C}^1$)

- Equality constraint

$$P_h(h(x)) = \frac{\mu_h}{2} h(x)^2$$

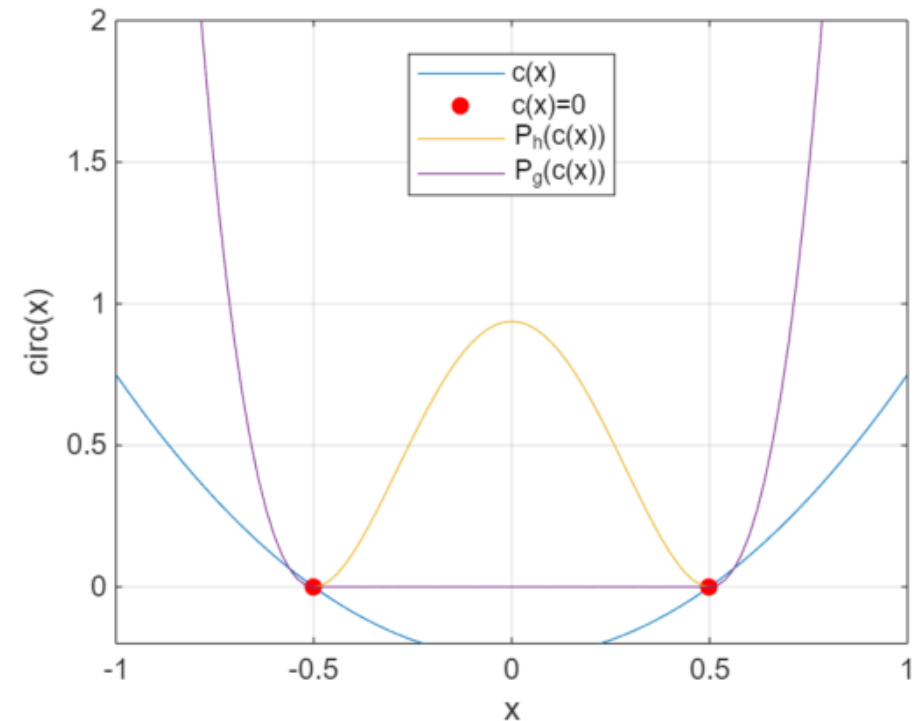
- Inequality constraint

$$P_g(g(x)) = \frac{\mu_g}{2} (\max\{0, g(x)\})^2$$

If several constraints:

$$P_h(x) = \sum_i \frac{\mu_i}{2} h_i(x)^2, \quad P_g = \sum_i \frac{\mu_g}{2} (\max\{0, g(x)\})^2$$

$$c(x) = x^2 - R^2$$



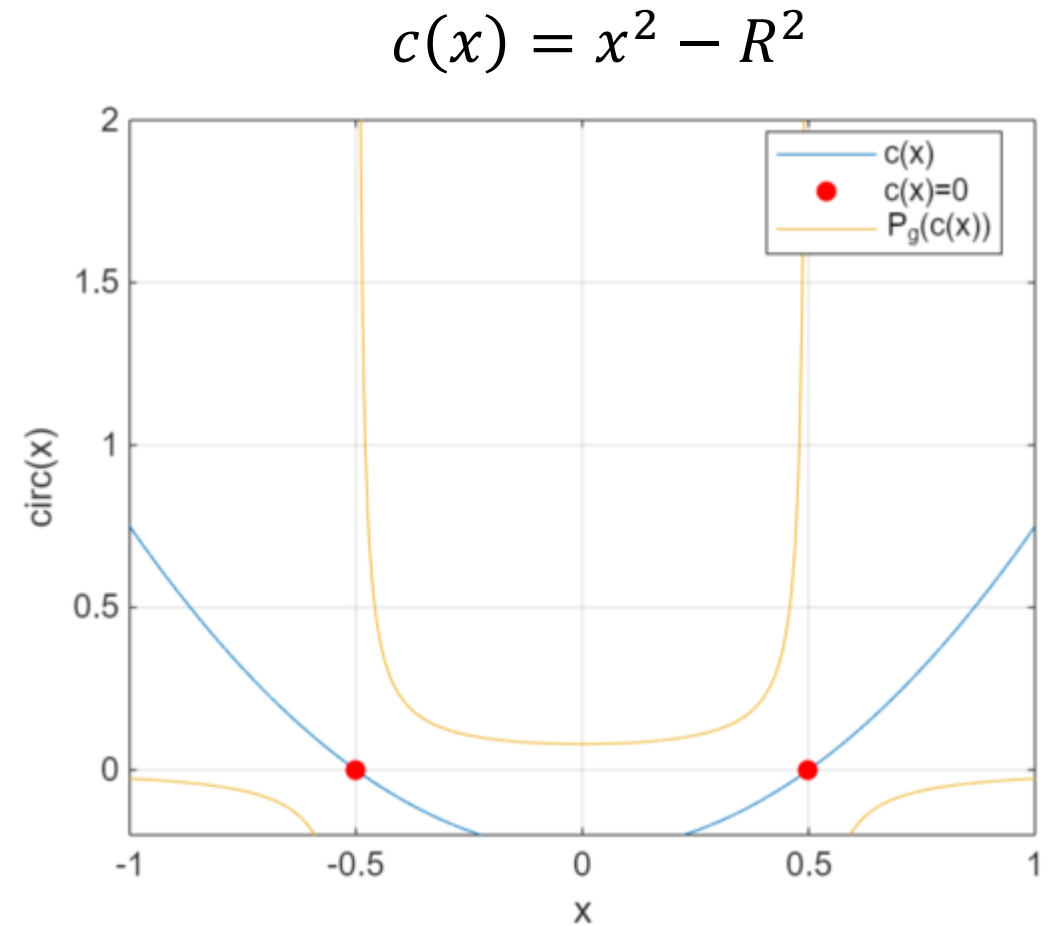
Penalty methods

Interior

Example of an interior penalty ($\mathcal{C}^1$)

- Equality constraint: **not suitable!**
- Inequality constraint:

$$P_g(x) = -\mu_g \frac{1}{g(x)}$$



Penalty methods

Problems

- Penalty methods are not accurate when μ_h and μ_g are fixed.
- If μ_h and μ_g are variable to converge exactly, then we may have a **ill-conditioned** problem
- We may use **continuation methods** (progressively increase μ)

Numerical example
Inductance optimization

Unconstrained methods & Penalty

Exercise

Problem solving – See website

Penalty methods

Exercises

Lagrangian

Dual problem

Let's recall the optimization problem (primal)

Minimize	$F(x)$	Objective function (cost)
Subject to	$h(x) = 0$	Equality constraint(s)
	$g(x) \leq 0$	Inequality constraint(s)

We define the **dual** problem as:

Maximize	$\min_x F(x) + \lambda \cdot h(x) + \nu \cdot g(x)$	Lagrangian
Subject to	$\nu \geq 0$	Inequality box constraint(s)

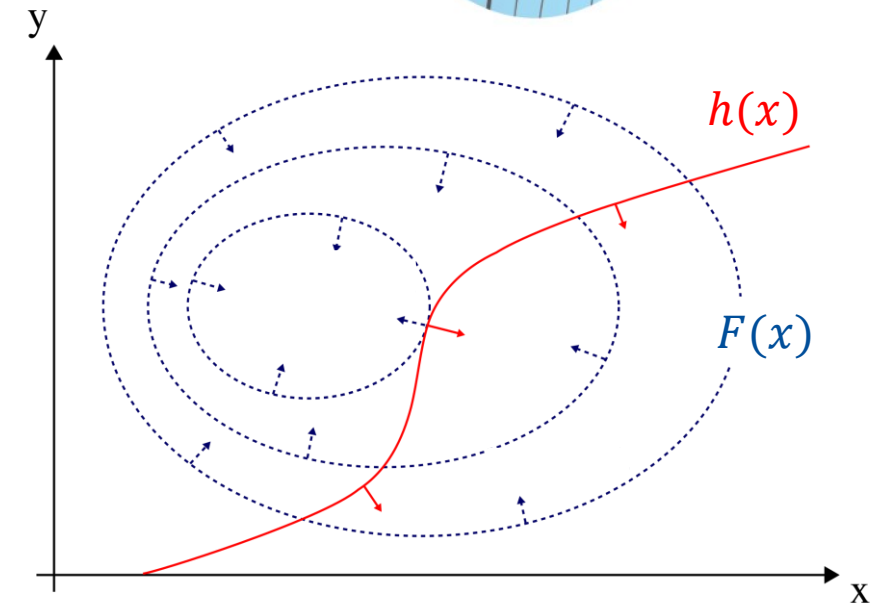
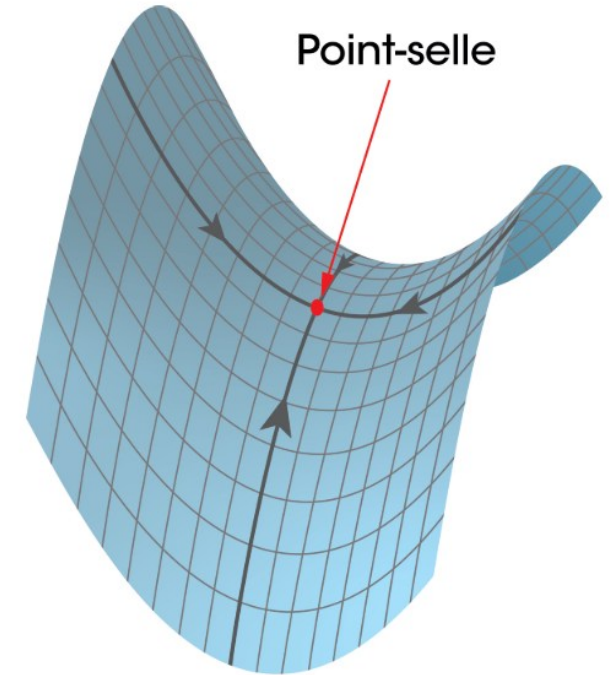
Lagrangian

KKT optimality condition

If $\mathcal{L}(x, \lambda, \nu) = F(x) + \lambda \cdot h(x) + \nu \cdot g(x)$ admits a saddle point, then solving the dual problem is equivalent to solving the primal problem.

Karush-Kuhn-Tucker optimality conditions

- Stationarity : $(\partial_x \mathcal{L}, \partial_\lambda \mathcal{L}, \partial_\nu \mathcal{L}) = (0, 0, 0)$
- Primal admissibility: $h(x) = 0, g(x) \leq 0$
- Dual admissibility: $\nu \geq 0$
- Complementarity : $\nu \cdot g(x) = 0$



Augmented Lagrangian

Principe

- We consider equality constraints (possible extension to inequalities). We write the Lagrangian, and we add a fixed penalization term

$$\mathcal{L}(x, \lambda) = F(x) + \lambda \cdot h(x) + \frac{\mu}{2} h(x)^2$$

We proceed in two stages (solving the dual problem) :

1. For a fixed λ , **minimization of $\mathcal{L}(x, \lambda)$** by a descent method
2. Once optimal x^* found, **update λ to maximize $\mathcal{L}$** depending on the constraint violation:

$$\lambda \leftarrow \lambda + \mu h(x^*)$$

And we start again until we converge. No need for $\mu \rightarrow \infty$!

SQP

Principle

- In the augmented Lagrangian, the "global" and "complicated" problem is solved each time.
- Another strategy is to solve a succession of "local" and "simple" problems (because they are approximated).
- For example, quadratic approximations, which can be solved using Newton's method.
- All that remains is to fulfill the constraints...

SQP

Principle

- Consider the local quadratic optimization problem

$$\text{Minimize} \quad F(x) = \frac{1}{2} x^T A x + b^T x$$

$$\text{Subject to} \quad h(x) = 0 \quad \text{Equality constraint(s)}$$

- Let us define the Lagrangian :

$$\mathcal{L}(x, \lambda) = F(x) + \lambda^T h(x)$$

If $h = A_h x + b_h$, then the condition of stationarity $(\partial_x \mathcal{L}, \partial_\lambda \mathcal{L}) = (0, 0)$ is :

$$\begin{bmatrix} A & A_h^T \\ A_h & 0 \end{bmatrix} \begin{bmatrix} x^* \\ \lambda^* \end{bmatrix} = \begin{bmatrix} -b \\ -b_h \end{bmatrix}$$

- And we can solve directly. If the constraints are not linear, we can apply Newton's method.
- We then start again in the vicinity of the next point; if the results are consistent with the local model, we extend the trust region.

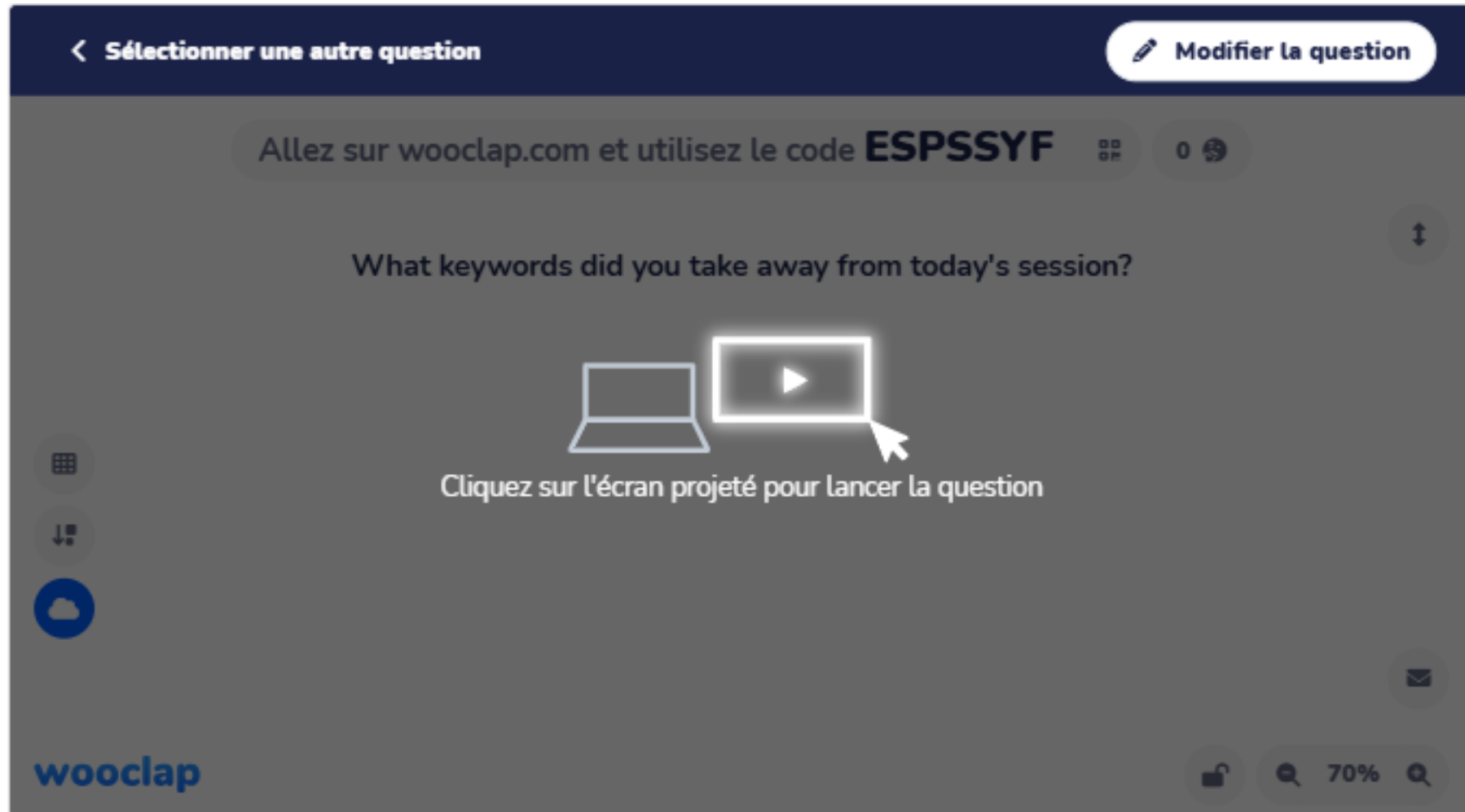
Dual methods

Exercises

CONCLUSION

What did you take away from the session?

Keywords



References

Courses, books, websites

- Lecture notes, W. Zulehner, Special Numerical Methods: Nonlinear Equations
- Nocedal, J., & Wright, S. J. (2006). Numerical optimization. In P. Glynn & S. M. Robinson (Eds.), *Springer Series in Operations Research*. Springer.
<https://doi.org/10.1201/b19115-11>
- Ern, A., & Allaire, G., Optimisation et Contrôle (French),
<http://www.cmap.polytechnique.fr/~allaire/map435/poly435.pdf>
- Cerf, M. Techniques d'optimization (French)
<http://dumas.perso.math.cnrs.fr/MINT-2018-TO1.pdf>

Thank you for your attention

Gradient optimization methods

Théodore Cherrière, Associate Professor CentraleSupélec, GeePs
theodore.cherriere@centralesupelec.fr